



Week 4 Tutorial: Data Collection & Annotation for Hand Gesture Recognition

Learning goals

By the end of this lab you will:

- Record your own RGB-D hand gesture clips using an Intel RealSense camera.
- Organise your data in the course-standard folder structure.
- Create binary hand masks for a subset of frames using a recommended annotation tool.
- Upload a dataset (in course-standard folder structure) on Moodle to pool into a shared COMP0248 hand gesture dataset.

Part 1: Data Collection – Hard Requirements – **Read carefully**

a. Capture the following 10 static gestures

G01_call 	G02_dislike 	G03_like 	G04_ok 	G05_one 
G06_palm 	G07_peace 	G08_rock 	G09_stop 	G10_three 

b. Data Contribution

- 10 predefined gestures × 5 clips/gesture = **50 clips**
- **Clip length:** 5 seconds (consider changing hand pose while keeping the gesture fixed during data recording)
- **Data capture modality:** RGB Depth (Real sense D455) – cameras only provided during the lab session
- **Annotation masks:** each student must provide binary hand masks for 2 keyframes per clip
 - Keyframes must be at least 1 sec apart and show complete hand in view.
 - Total annotations: 2 x 50 clips = 100 annotated frames per student

c. Data Constraints:

To control data variability, following constraints must be considered during data capturing:
(Use the provided script for data collection)

- Camera: Intel RealSense RGB-D (D455)
- Resolution: 640×480 (fixed for all students)
- FPS: 3
- Capture data using only right hand (other hand must be out of frame)
- Face out of frame (shoulders allowed)
- Hand distance: roughly 60 – 100 cm from D455 (one arm left) – ensure depth map looks reliable.
- Hand should not be truncated; keep entire hand visible and possibly centred
- Select five different backgrounds for the 5 set of recordings.
 - E.g., uncluttered or cluttered background, varying illumination (in dark or bright room) – justify your background choice

d. Data folder structure

For each student, the collected and annotated data must be stored in the following folder structure, zipped and uploaded on the provided Moodle link.

dataset/

```
└─ <studentno>_<surname>/
  └─ G01_call/
    │ └─ clip01/
    │ │ └─ rgb/
    │ │ │ └─ frame_001.png
    │ │ │ └─ frame_002.png
    │ │ │ └─ ...
    │ │ └─ depth/
    │ │ │ └─ frame_001.png
    │ │ │ └─ frame_002.png
    │ │ │ └─ ...
    │ │ └─ depth_raw/
    │ │ │ └─ frame_001.npy
    │ │ │ └─ frame_002.npy
    │ │ │ └─ ...
    │ │ └─ annotation/
    │ │ │ └─ frame_00X.png # only keyframes have masks (00X to be the same number as in the rgb folder)
    │ │ │ └─ frame_00Y.png
    │ │ │ └─ ...
    │ │ └─ depth_metadata.json
    └─ clip02/
      │ └─ rgb/
      │ └─ depth/
      │ └─ depth_raw/
      │ └─ annotation/
      │ └─ depth_metadata.json
    └─ clip03/
    └─ clip04/
```

- | └─ clip05/
- └─ G02_dislike/
- | └─ clip01/
- | └─ clip02/
- | └─ clip03/
- | └─ clip04/
- | └─ clip05/
- └─ G03_like/
- | └─ clip01/
- | └─ clip02/
- | └─ clip03/
- | └─ clip04/
- | └─ clip05/
- └─ G04_ok/
- | └─ clip01/
- | └─ ... (clip02–clip05)
- └─ G05_one/
- | └─ ... (clip01–clip05)
- └─ G06_palm/
- | └─ ... (clip01–clip05)
- └─ G07_peace/
- | └─ ... (clip01–clip05)
- └─ G08_rock/
- | └─ ... (clip01–clip05)
- └─ G09_stop/
- | └─ ... (clip01–clip05)
- └─ G10_three/
- └─ ... (clip01–clip05)

Part 2: Data Collection using Intel RealSense

Data recording with RealSense cameras:

- Use the code ([stream_realsense_capture.py](#)) provided with this guide for the data collection. **You could modify the code (as per your need) as long as the above requirements are met.**
- The provided code captures a frame at 1280 x 720, crops (both RGB and depth) to centre (while preserving the aspect ratio) to 640 x 360.
- Frames are captured at 3 fps.
- The code allows capturing one set (1 scenario) of 10 gestures.

Part 3: Data Annotation (suggested approach)

Step Zero: Setup for Annotation

Download the associated scripts ([annotation_scripts.zip](#)) and place in the directory containing *dataset/*.

Before running scripts: *pip install label-studio-converter*

Scripts: **get_all_frames.py** & **process_LS_output.py** require filename specifications to match your dataset names

Step One: Select frames for labelling

Run the **get_all_frames.py** script to copy all rgb frames to a new folder – this will construct a new folder within dataset which can be used temporarily for annotation

```
dataset/  
    labelling/  
        images/  
        Labels/
```

Select frames in labelling/images for annotation and remove any irrelevant frames – this should leave you with 100 rgb frames to annotate.

Step Two: Pre-annotate with SAM3 – Google Colab

Upload the **sam3_annotation.ipynb** notebook to Google Colab and follow the instructions outlined to generate preliminary annotations for your dataset. After completing this, download the resulting labels.zip file, unzip the contents, and copy the mask .png files into *dataset/labelling/labels*.

Step Three: Refine annotations with Label Studio

To add, refine, and correct masks, we use LabelStudio, an open-source annotation platform.

Convert Annotations

We must first convert the existing mask labels to a compatible format using the **convert_annotations_for_LS.py** script which should produce *labels.json* in *dataset/labelling/*.

[Install Label Studio](#) & [Configure Local Storage](#)

Execute in command prompt/terminal:

```
>> python -m pip install label-studio  
>> set LABEL_STUDIO_LOCAL_FILES_SERVING_ENABLED=true  
>> set LABEL_STUDIO_LOCAL_FILES_DOCUMENT_ROOT=<PATH TO dataset/>
```

Check setup: should return your entered values, otherwise re-start cmd window and check again

```
>> echo %LABEL_STUDIO_LOCAL_FILES_SERVING_ENABLED%
```

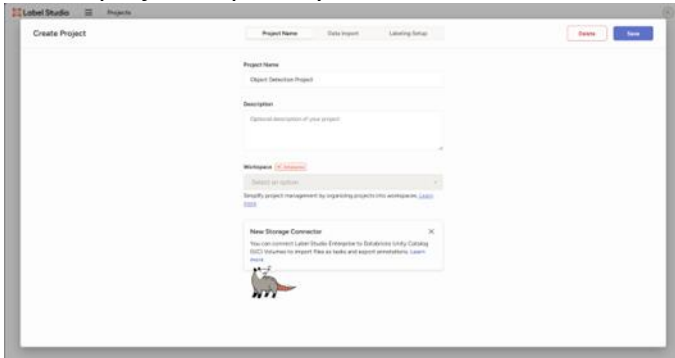
```
>> echo %LABEL_STUDIO_LOCAL_FILES_DOCUMENT_ROOT%
```

Launch label studio

```
>> label-studio
```

Setup Project

Create project, input any name.



Set labelling interface using the code tab, paste the following as shown below:

<View>

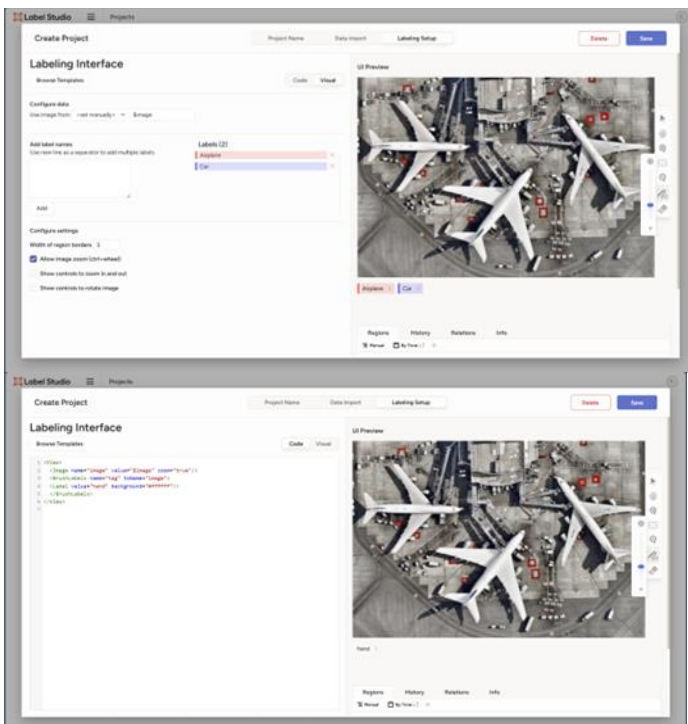
```
<Image name="image" value="$image" zoom="true"/>
```

```
<BrushLabels name="tag" toName="image">
```

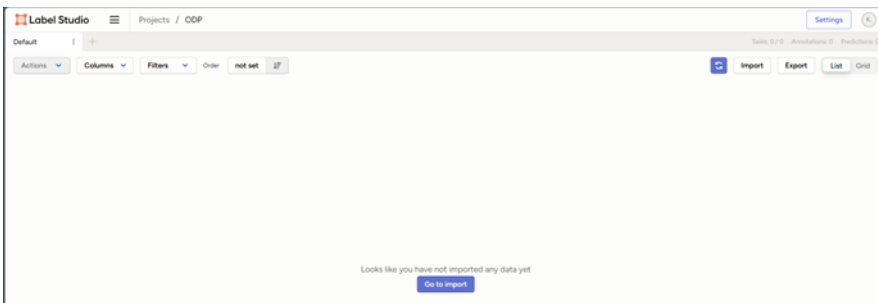
```
<Label value="hand" background="#ffffff"/>
```

```
</BrushLabels>
```

</View>

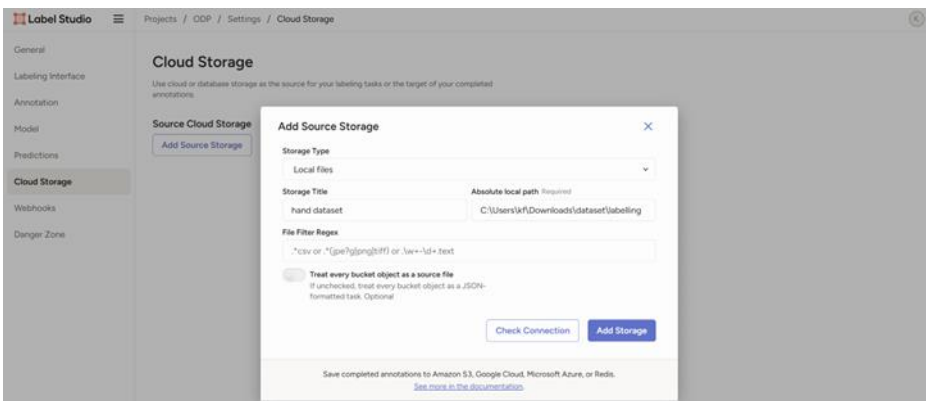


Skip the Data Import tab. Press **save** which will open a project with nothing present.

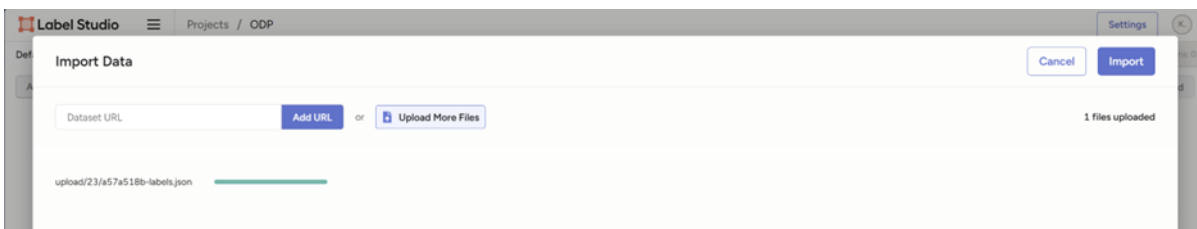


Now click **settings** then **cloud storage** and press **add source storage**.

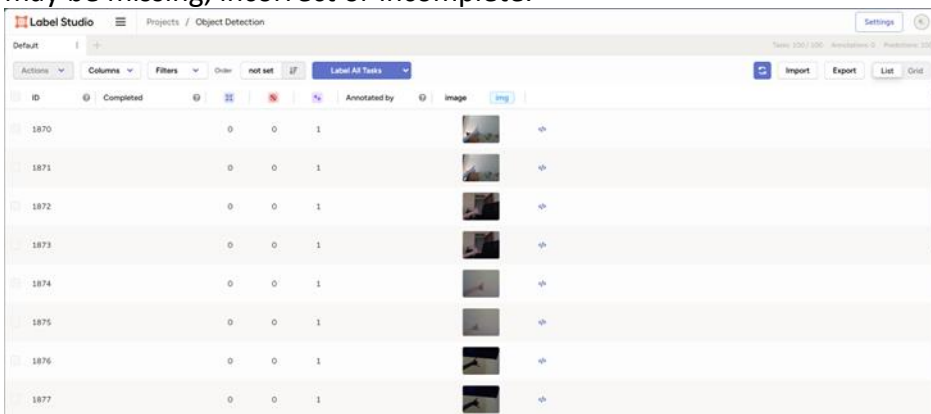
Configure source storage to “Local Files”, name the dataset anything you’d like, and set the absolute local path to be the path to the temporary labelling directory. Click **add storage**, **do not** click sync storage when the window closes or you’ll have duplicate images.



Navigate back to the main project page by clicking the project name at the top of the page. Then press **go to import** and import the *labels.json* file produced earlier.

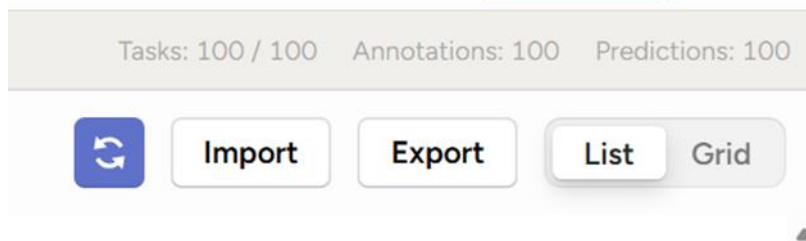


Once you click **import**, you should now have your images for annotations loaded with any masks generated by SAM3. You can now work through with a brush tool to fix or add any annotations which may be missing, incorrect or incomplete.

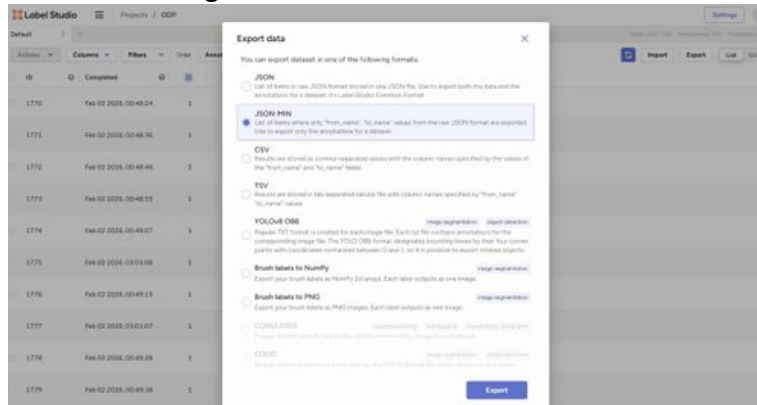


Save final masks

Once you have edited your masks and are happy with your dataset, you need to check all tasks are marked as complete before exporting as only completed task labels will be exported. This can be completed by clicking label all tasks and submitting all masks even if no changes were required.



Once this is done, click export on the main page, exporting the labels to “JSON-MIN” which will download a single file.



Once this is downloaded, move to the parent directory of *dataset/* and rename to “*annotations.json*”. You can then run *process_LS_output.py* to convert the JSON annotations to masks and file them in the existing file directory.

You should now have a copy of all your finished annotations within each endpoint *annotations/* folder. You should now remove any additional folders and files which were used for this process before uploading your dataset as per the specified